

INFORME TÉCNICO — TALLER SEMANA 9 CRM "Gestión Integral de Negocios"

Docente: Ing. José Daniel Cadenas L. Fecha: 30/09/2026



Datos del estudiante

Nombre y apellido: Sabrina Hernández **Cédula:** V31371366

Sección: B

Grupo N°: **Correo institucional:** sabrinasejh@gmail.com

Repositorio GitHub: https://github.com/xS4b/Taller_Laravel9



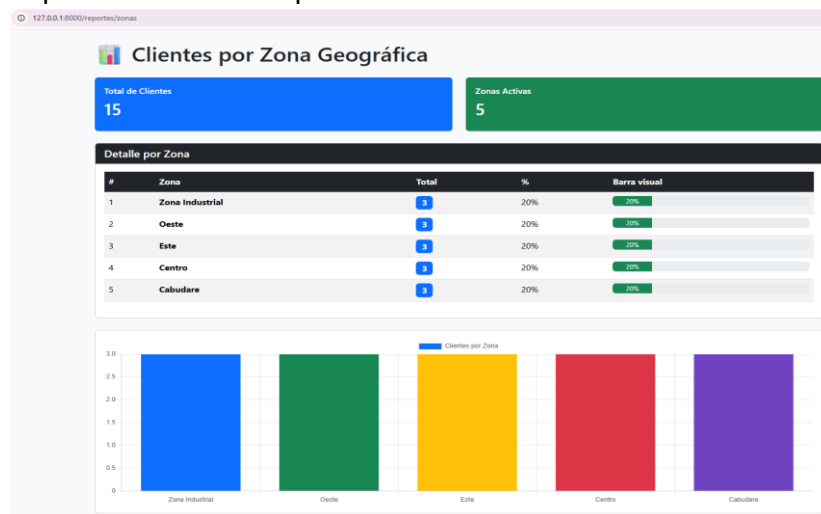
Asignación de preguntas de defensa

Los 3 últimos dígitos de mi cédula son: 3 6 6, la suma de estos dígitos es: 15, el resultado de (suma % 10) es: 1.5 y Mi SET de preguntas asignado es: SET B



EVIDENCIA 1 — Capturas de los 2 reportes (2%)

Reporte 1 — Clientes por Zona:



Reporte 2 — Interacciones por Asesor:



✓ EVIDENCIA 2 — Respuestas a las 6 preguntas de comprensión (3%)

Respuesta 1:

Aparecen tanto los clientes con zona geografica 'este' como 'oeste'

Respuesta 2:

Sin 'group by' generaría un error ya que zona_geografica es una sola columna y count lo que hace es comprimir filas

Respuesta 3:

Carlos tiene 0 clientes, no hay registros en la tabla de clientes que se vinculen a su ID. El sistema simplemente lo descarta del resultado final.

Asumiendo que la tabla de asesores está a la izquierda de la consulta, el LEFT JOIN garantiza que ningún asesor se quede fuera, tenga o no clientes

Respuesta 4:

En un reporte de interacciones por asesor, la decisión de usar LEFT JOIN en lugar de INNER JOIN responde a una necesidad clave de negocio: evitar perder información sobre el desempeño global del equipo de asesores.

Respuesta 5:

Se utiliza para insertar expresiones de SQL nativo o personalizado dentro de las consultas del Query Builder o Eloquent sin que el framework intente escapar o transformar esos fragmentos de código.

Respuesta 6: : El get devuelve una lista con todos los registros que encuentre y first solo muestra el primer registro que encuentre.

✓ EVIDENCIA 3 — Código comentado de UN método (2%)

```
public function interaccionesPorAsesor()
{
    // Saco los datos de los asesores con la cantidad de interacciones
    $asesores = DB::table('users') // ← HUECO A
        ->select(
            'users.id', 'users.name',
            // Cuenta las llamadas
            DB::raw("COUNT(CASE WHEN interactions.tipo_interaccion = 'Llamada'
            THEN 1 END) AS llamadas"), // ← HUECO B
            // Cuenta las visitas
            DB::raw("COUNT(CASE WHEN interactions.tipo_interaccion = 'Visita'
            THEN 1 END) AS visitas"), // ← HUECO C
            // Cuenta los WhatsApp
        )
}
```

```

        DB::raw("COUNT(CASE WHEN interactions.tipo_interaccion = 'WhatsApp'
THEN 1 END) AS whatsapp"), // ← HUECO D
        // Cuenta el total de interacciones
        DB::raw('COUNT(interactions.id) AS total') // ← HUECO E
    )
    // Uno los asesores con sus clientes
    ->leftJoin('clients', 'clients.user_id', '=', 'users.id') //← HUECO F
    // Uno los clientes con sus interacciones
    ->leftJoin('interactions', 'interactions.client_id', '=',
'clients.id') // ← HUECO G
    // Agrupo por asesor para que cuente por cada uno
    ->groupBy('users.id', 'users.name') // ← HUECO H, I
    // Ordeno del que tiene más interacciones al que tiene menos
    ->orderByDesc('total') // ← HUECO J
    ->get();
    // Saco los nombres para el gráfico
    $labels = $asesores->pluck('name')->toArray(); // ← HUECO K
    // Saco las llamadas, visitas y whatsapp para el gráfico
    $llamadas = $asesores->pluck('llamadas')->toArray(); // ← HUECO L
    $visitas = $asesores->pluck('visitas')->toArray(); // ← HUECO M
    $whatsapp = $asesores->pluck('whatsapp')->toArray(); // ← HUECO N

    // Mando todo a la vista
    return view('reports.interacciones', compact(
        'asesores', 'labels', 'llamadas', 'visitas', 'whatsapp'
    ));
}

```

✅ EVIDENCIA 4 — Retos de modificación resueltos (2%) Reto 1 (filtro > 15

```

// reto 1
    $zonasConPorcentaje = $zonasConPorcentaje->filter(function ($zona) {
        return $zona->porcentaje > 15;
    })->values();

```

Reto 2 (desglose por día):

✓ EVIDENCIA 5 — Reflexión breve (100 palabras)

¿Qué concepto fue más difícil?

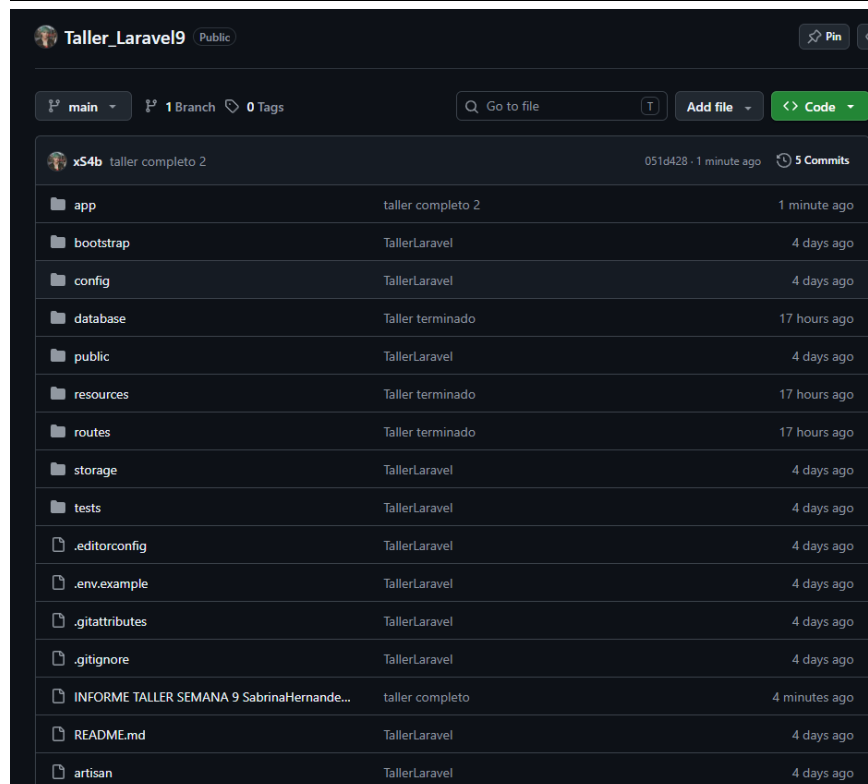
Lo que me pareció más difícil fue no saber exactamente donde iba cada cosa en el código, como hay tantos apartados, carpetas y archivos regados por todos lados, uno se marea rapido tratando de adivinar dónde poner cada función o línea sin terminar rompiendo el programa entero

¿Cómo aplicarás esto al CRM final?

Esto lo voy a aplicar en el crm final intentando ser mucho más ordenado desde el principio, separando bien cada parte y teniendo más cuidado con la estructura para no perder tanto tiempo buscando sin saber cuando algo falle

✓ EVIDENCIA 6 — Commit en GitHub

```
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 926 bytes | 926.00 KiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
To https://github.com/xS4b/Taller_Laravel9
601a9e8..051d428  main -> main
```



The screenshot shows the GitHub interface for the repository 'Taller_Laravel9' (Public). The repository has 1 Branch (main) and 0 Tags. The file structure is listed below:

File/Folder	Author	Commit Time
app	taller completo 2	1 minute ago
bootstrap	TallerLaravel	4 days ago
config	TallerLaravel	4 days ago
database	Taller terminado	17 hours ago
public	TallerLaravel	4 days ago
resources	Taller terminado	17 hours ago
routes	Taller terminado	17 hours ago
storage	TallerLaravel	4 days ago
tests	TallerLaravel	4 days ago
.editorconfig	TallerLaravel	4 days ago
.env.example	TallerLaravel	4 days ago
.gitattributes	TallerLaravel	4 days ago
.gitignore	TallerLaravel	4 days ago
INFORME TALLER SEMANA 9 SabrinaHernande...	taller completo	4 minutes ago
README.md	TallerLaravel	4 days ago
artisan	TallerLaravel	4 days ago